Ushtrime
Fjalorë dhe Set

Alion Emini

kurseinformatike.com

Përmbajtja

1. Pyetje me alternativa
2. Pyetje True/False
3. Pyetje të shkurtra
4. Algoritme
5. Ushtrime në kod

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Multiple Choice #1

Pyetja. Cilin operator përdor për të parë nëse një çelës ekziston në një fjalor?

a) `&`

b) `in`

c) `^`

d) `?`

Multiple Choice #1 - Zgjidhja

Zgjidhja: b) `in`

Operatori `in` kontrollon ekzistencën e çelësit: `'x' in dct`.

Multiple Choice #2

Pyetja. Me çfarë fshin një element nga një fjalor?

a) metoda `remove`

b) metoda `erase`

c) metoda `delete`

d) deklarata `del`

Multiple Choice #2 - Zgjidhja

Zgjidhja: d) `del`

`del dct[key]` ose `dct.pop(key)`.

Multiple Choice #3

Pyetja. Cila funksion kthen numrin e elementeve në një fjalor?

a) `size()`

b) `len()`

c) `elements()`

d) `count()`

Multiple Choice #3 - Zgjidhja

Zgjidhja: b) `len()`

Kthen numrin e çifteve *keyvalue*.

Multiple Choice #4

Pyetja. Si krijon një fjalor bosh?

a) `{}`

b) `()`

c) `[]`

d) `empty()`

Multiple Choice #4 - Zgjidhja

Zgjidhja: a) `{}`
Edhe `dict()` funksionon.

Multiple Choice #5

Pyetja. Cila metodë kthen një çift çelësvlerë dhe e heq atë nga fjalori?

a) `pop()`

b) `random()`

c) `popitem()`

d) `rand_pop()`

Multiple Choice #5 - Zgjidhja

Zgjidhja: c) `popitem()`

`popitem()` heq *dhe* kthen një çift (në Python modern: të fundit të futur).

Multiple Choice #6

Pyetja. Cila metodë kthen vlerën e një çelësi dhe e heq atë element?

a) `pop()`

b) `random()`

c) `popitem()`

d) `rand_pop()`

Multiple Choice #6 - Zgjidhja

Zgjidhja: a) `pop()`

Multiple Choice #7

Pyetja. Cila metodë e fjalorit kthen vlerën e një çelësi ose një vlerë parazgjedhje nëse sgjendet?

a) `pop()`
b) `key()`
c) `value()`
d) `get()`

Multiple Choice #7 - Zgjidhja

Zgjidhja: d) `get()`

Multiple Choice #8

Pyetja. Cila metodë kthen të gjithë çelësat me vlerat e tyre si një sekuencë `tuple`-sh?

a) `keys_values()`

b) `values()`

c) `items()`

d) `get()`

Multiple Choice #8 - Zgjidhja

Zgjidhja: c) `items()`

Multiple Choice #9

Pyetja. Cila funksion kthen numrin e elementeve në një `set`?

a) `size()`

b) `len()`

c) `elements()`

d) `count()`

Multiple Choice #9 - Zgjidhja

Zgjidhja: b) `len()`

Multiple Choice #10

Pyetja. Me cilën metodë shton *një* element në një `set`?

a) `append`

b) `add`

c) `update`

d) `merge`

Multiple Choice #10 - Zgjidhja

Zgjidhja: b) `add`

Multiple Choice #11

Pyetja. Me cilën metodë shton një grup elementesh në një `set`?

a) `append`

b) `add`

c) `update`

d) `merge`

Multiple Choice #11 - Zgjidhja

Zgjidhja: c) `update`

Multiple Choice #12

Pyetja. Cila metodë heq një element por *nuk* hedh përjashtim nëse sgjendet?

a) `remove`

b) `discard`

c) `delete`

d) `erase`

Multiple Choice #12 - Zgjidhja

Zgjidhja: b) `discard`

Multiple Choice #13

Pyetja. Cila metodë heq një element dhe hedh përjashtim nëse sgjendet?

a) `remove`

b) `discard`

c) `delete`

d) `erase`

Multiple Choice #13 - Zgjidhja

Zgjidhja: a) `remove`

Multiple Choice #14

Pyetja. Operatori për bashkimin e dy `set`-eve:

a) `|`

b) `&`

c) `-`

d) `^`

Multiple Choice #14 - Zgjidhja

Zgjidhja: a) |

Multiple Choice #15

Pyetja. Operatori për diferencën e dy `set`-eve:

a) `|`

b) `&`

c) `-`

d) `^`

Multiple Choice #15 - Zgjidhja

Zgjidhja: c) –

Multiple Choice #16

Pyetja. Operatori për prèrjen (ndërprerjen) e dy `set`-eve:

a) `|`

b) `&`

c) `-`

d) `^`

Multiple Choice #16 - Zgjidhja

Zgjidhja: b) `&`

Multiple Choice #17

Pyetja. Operatori për diferencën simetrike të dy `set`-eve:

a) `|`

b) `&`

c) `-`

d) `^`

Multiple Choice #17 - Zgjidhja

Zgjidhja: d) ^

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

True / False #1

Çelësat në një fjalor duhet të jenë objekte të ndryshueshme.

True / False #1 - Zgjidhja

Zgjidhja: False. Duhet të jenë *të pandryshueshëm* (p.sh. `str`, `int`, `tuple`).

True / False #2

Fjalorët nuk janë sekuenca.

True / False #2 - Zgjidhja

Zgjidhja: True. Janë *mapping*, jo sekuenca.

True / False #3

Një `tuple` mund të jetë çelës i një fjalori.

True / False #3 - Zgjidhja

Zgjidhja: True.

True / False #4

Një `list` mund të jetë çelës i një fjalori.

True / False #4 - Zgjidhja

Zgjidhja: False. `list` është mutable.

True / False #5

`popitem` nuk hedh përjashtim nëse fjalori është bosh.

True / False #5 - Zgjidhja

Zgjidhja: False. Hedh `KeyError`.

True / False #6

`mydct = {}` krijon një fjalor bosh.

True / False #6 - Zgjidhja

Zgjidhja: True.

True / False #7

`myset = ()` krijon një `set` bosh.

True / False #7 - Zgjidhja

Zgjidhja: False. Kjo është *tuple* bosh; përdor `set()`.

True / False #8

`set`-et i ruajnë elementet pa renditje.

True / False #8 - Zgjidhja

Zgjidhja: True.

True / False #9

Mund të ruash dublikate në një `set`.

True / False #9 - Zgjidhja

Zgjidhja: False.

True / False #10

`remove` hedh përjashtim nëse elementi mungon.

True / False #10 - Zgjidhja

Zgjidhja: True. (ndryshe `discard`).

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Pyetje të shkurtra #1

Pyetja. Çfarë do të shfaqë kodi i mëposhtëm?

```
dct = {'Monday':1, 'Tuesday':2, 'Wednesday':3}
print(dct['Tuesday'])
```

Pyetje të shkurtra #1 - Zgjidhja

Përgjigja: 2

Pyetje të shkurtra #2

Pyetja. Çfarë do të shfaqë kodi i mëposhtëm?

```
dct = {'Monday':1, 'Tuesday':2, 'Wednesday':3}
print(dct.get('Monday', 'Not found'))
```

Pyetje të shkurtra #2 - Zgjidhja

Përgjigja: 1

Pyetje të shkurtra #3

Pyetja. Çfarë do të shfaqë kodi i mëposhtëm?

```
dct = {'Monday':1, 'Tuesday':2, 'Wednesday':3}
print(dct.get('Friday', 'Not found'))
```

Pyetje të shkurtra #3 - Zgjidhja

Përgjigja: `Not found`

Pyetje të shkurtra #4

Pyetja. Çfarë do të shfaqë kodi i mëposhtëm?

```
stuff = {'aaa':111, 'bbb':222, 'ccc':333}
print(stuff['bbb'])
```

Pyetje të shkurtra #4 - Zgjidhja

Përgjigja: 222

Pyetje të shkurtra #5

Pyetja. Si fshihet një element nga një fjalor?

```
# shembull
dct = {'a':1, 'b':2}
# ...
```

Pyetje të shkurtra #5 - Zgjidhja

Përgjigja: `del dct['a']` (ose `dct.pop('a')`).

Pyetje të shkurtra #6

Pyetja. Si gjen numrin e elementeve në një fjalor?

```
# ...
```

Pyetje të shkurtra #6 - Zgjidhja

Përgjigja: `len(dct)`

Pyetje të shkurtra #7

Pyetja. Çfarë do të shfaqë kodi?

```
dct = {1:[0,1], 2:[2,3], 3:[4,5]}
print(dct[3])
```

Pyetje të shkurtra #7 - Zgjidhja

Përgjigja: `[4, 5]`

Pyetje të shkurtra #8

Pyetja. Cilat vlera do të shfaqen (rendi i papërcaktuar)?

```
dct = {1:[0,1], 2:[2,3], 3:[4,5]}
for k in dct:
    print(k)
```

Pyetje të shkurtra #8 - Zgjidhja

Përgjigja: `1, 2, 3` (rend i papërcaktuar).

Pyetje të shkurtra #9

Pyetja. Çfarë elementesh do të përmbajë `myset` pas ekzekutimit?

```
1 myset = set('Saturn')
```

Pyetje të shkurtra #9 - Zgjidhja

Përgjigja: {'S','a','t','u','r','n'}

Pyetje të shkurtra #10

Pyetja. Çfarë ndodh me këtë deklaratë?

```
myset = set(10)
```

Pyetje të shkurtra #10 - Zgjidhja

Përgjigja: **TypeError** (numri sështë iterable).

Pyetje të shkurtra #11

Pyetja. Çfarë elementesh do të ruhen?

```
myset = set('a bb ccc dddd')
```

Pyetje të shkurtra #11 - Zgjidhja

Përgjigja: karaktere unike (edhe hapësira): {`'a',' ','b','c','d'`}.

Pyetje të shkurtra #12

Pyetja. Çfarë elementesh do të ruhen?

```
myset = set([2,4,4,6,6,6,6])
```

Pyetje të shkurtra #12 - Zgjidhja

Përgjigja: $\{2, 4, 6\}$.

Pyetje të shkurtra #13

Pyetja. Çfarë elementesh do të ruhen?

```
myset = set(['a','bb','ccc','dddd'])
```

Pyetje të shkurtra #13 - Zgjidhja

Përgjigja: {'a','bb','ccc','dddd'}.

Pyetje të shkurtra #14

Pyetja. Çfarë do të shfaqë kodi?

```
myset = set('1 2 3')
print(len(myset))
```

Pyetje të shkurtra #14 - Zgjidhja

Përgjigja: 4 (karakteret `'1','  ','2','3'`).

Pyetje të shkurtra #15

Pyetja. Cilët elementë do të ketë set3?

```
set1 = set([10,20,30,40])
set2 = set([40,50,60])
set3 = set1.union(set2)
```

Pyetje të shkurtra #15 - Zgjidhja

Përgjigja: $\{10, 20, 30, 40, 50, 60\}$.

Pyetje të shkurtra #16

Pyetja. Cilët elementë do të ketë set3?

```
set1 = set(['o','p','s','v'])
set2 = set(['a','p','r','s'])
set3 = set1.intersection(set2)
```

Pyetje të shkurtra #16 - Zgjidhja

Përgjigja: {'p','s'}.

Pyetje të shkurtra #17

Pyetja. Cilët elementë do të ketë set3?

```
set1 = set(['d','e','f'])
set2 = set(['a','b','c','d','e'])
set3 = set1.difference(set2)
```

Pyetje të shkurtra #17 - Zgjidhja

Përgjigja: {'f'}.

Pyetje të shkurtra #18

Pyetja. Cilët elementë do të ketë set3?

```
set1 = set(['d','e','f'])
set2 = set(['a','b','c','d','e'])
set3 = set2.difference(set1)
```

Pyetje të shkurtra #18 - Zgjidhja

Përgjigja: {'a','b','c'}.

Pyetje të shkurtra #19

Pyetja. Cilët elementë do të ketë set3?

```
set1 = set([1,2,3])
set2 = set([2,3,4])
set3 = set1.symmetric_difference(set2)
```

Pyetje të shkurtra #19 - Zgjidhja

Përgjigja: $\{1, 4\}$.

Pyetje të shkurtra #20

Pyetja. *Subset* dhe *superset*?

```
set1 = set([100,200,300,400,500])
set2 = set([200,400,500])
```

Pyetje të shkurtra #20 - Zgjidhja

Përgjigja: `set2` është *subset* i `set1`; `set1` është *superset* i `set2`.

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Algoritmi #1

Krijo një fjalor me çiftet: `'a':1, 'b':2, 'c':3`

Algoritmi #1 - Zgjidhja

```
dct = {'a': 1, 'b': 2, 'c': 3}
```

Algoritmi #2

Krijo një fjalor bosh.

Algoritmi #2 - Zgjidhja

```
dct = {}
```

Algoritmi #3

Nëse `'James'` është në `dct`, shfaq vlerën; përndryshe shfaq `'Not found'`.

Algoritmi #3 - Zgjidhja

```
if 'James' in dct:
    print(dct['James'])
else:
    print('Not found')
```

Algoritmi #4

Nëse `'Jim'` është në `dct`, fshije atë me vlerën përkatëse.

Algoritmi #4 - Zgjidhja

```
if 'Jim' in dct:
    del dct['Jim']
```

Algoritmi #5

Krijo një `set` me anëtarët {10, 20, 30, 40}.

Algoritmi #5 - Zgjidhja

```
s = {10, 20, 30, 40}
```

Algoritmi #6

Krijo `set3` si bashkimi i `set1` dhe `set2`.

Algoritmi #6 - Zgjidhja

```
set3 = set1 | set2
```

Algoritmi #7

Krijo `set3` si prèrja (intersection) e `set1` dhe `set2`.

Algoritmi #7 - Zgjidhja

```
set3 = set1 & set2
```

Algoritmi #8

Krijo `set3` me elementët që janë në `set1` por jo në `set2`.

Algoritmi #8 - Zgjidhja

```
set3 = set1 - set2
```

Algoritmi #9

Krijo `set3` me elementët që janë në `set2` por jo në `set1`.

Algoritmi #9 - Zgjidhja

```
set3 = set2 - set1
```

Algoritmi #10

Krijo `set3` me elementët që nuk janë të përbashkët (symmetric difference).

Algoritmi #10 - Zgjidhja

```
set3 = set1 ^ set2
```

Algoritmi #11

Përdor *dictionary comprehension*: nga lista `numbers` krijo `{n: n*10}`.

Algoritmi #11 - Zgjidhja

```
numbers = [1, 2, 3, 4, 5]
dct = {n: n*10 for n in numbers}
```

Algoritmi #12

Nga `test_averages` filtro në `high_scores` vetëm ata me vlerë ≥ 90.

Algoritmi #12 - Zgjidhja

```
high_scores = {k: v for k, v in test_averages.items() if v >= 90}
```

Algoritmi #13

Pickle: ruaj fjalorin `dct` në skedarin `mydata.dat`.

Algoritmi #13 - Zgjidhja

```
import pickle

with open('mydata.dat', 'wb') as f:
    pickle.dump(dct, f, protocol=pickle.HIGHEST_PROTOCOL)
```

Algoritmi #14

Unpickle: lexo nga `mydata.dat` dhe rikthe `dct`.

Algoritmi #14 - Zgjidhja

```
import pickle

with open('mydata.dat', 'rb') as f:
    dct = pickle.load(f)
```

1 Pyetje me alternativa

2 Pyetje True/False

3 Pyetje të shkurtra

4 Algoritme

5 Ushtrime në kod

Ushtrimi #1 - Informacione kursesh I

Shkruani një program që krijon tre **fjalorë** (`dict`) për kurset:

- Fjalor që lidh **numrin e kursit** me **numrin e dhomës** ku zhvillohet kursi.
- Fjalor që lidh **numrin e kursit** me **pedagogun** që e zhvillon kursin.
- Fjalor që lidh **numrin e kursit** me **orarin** e kursit.

Dhoma e kursit

Kursi	Dhoma
CS101	3004
CS102	4501
CS103	6755
NT110	1244
CM241	1411

Pedagogu

Kursi	Pedagogu
CS101	Haynes
CS102	Alvarado
CS103	Rich
NT110	Burke
CM241	Lee

Orari

Kursi	Orari
CS101	8:00 a.m.
CS102	9:00 a.m.
CS103	10:00 a.m.
NT110	11:00 a.m.
CM241	1:00 p.m.

Ushtrimi #1 - Informacione kursesh II

Kërkohet: Programi duhet t'i lejojë përdoruesit të fusë një numër kursi (p.sh. `CS101`) dhe të shfaqë **numrin e dhomës**, **pedagogun** dhe **orarin** e atij kursi.

Ushtrimi #1 - Zgjidhja

```
def main():
    rooms = {'CS101':3004, 'CS102':4501, 'CS103':6755, 'NT110':1244, 'CM241':1411}
    instructors = {'CS101':'Haynes', 'CS102':'Alvarado', 'CS103':'Rich', 'NT110':'Burke', 'CM241':'Lee'}
    times = {'CS101':'8:00 am', 'CS102':'9:00 am', 'CS103':'10:00 am', 'NT110':'11:00 am', 'CM241':'12:00 pm'}

    course = input('Enter a course number: ')

    if course not in rooms:
        print(f'{course} is an invalid course number.')
    else:
        print(f'The details for course {course} are:')
        print(f'Room: {rooms[course]}')
        print(f'Instructor: {instructors[course]}')
        print(f'Time: {times[course]}')

if __name__ == '__main__':
    main()
```

Ushtrimi #2 Kuizi i kryeqyteteve

Shkruani një program që:

- krijon një **fjalor** ku çelësat janë emrat e **shteteve** (p.sh. shtetet e SHBA-së) dhe vlerat janë **kryeqytetet** e tyre;
- përdor Internetin (ose një burim tjetër) për të ndërtuar listën e shteteve dhe kryeqyteteve;
- zgjedh **rastësisht** një shtet dhe i kërkon përdoruesit të shkruajë kryeqytetin përkatës;
- kontrollon nëse përgjigjja është e saktë duke përdorur fjalorin;
- mban numrin e **përgjigjeve të sakta** dhe **të gabuara**;
- në fund shfaq përmbledhjen: sa të sakta dhe sa të gabuara.

Shënim: Në vend të shteteve të SHBA-së mund të përdorësh edhe emrat e **vendeve** dhe kryeqytetet e tyre.

Ushtrimi #2 - Zgjidhja I

```
import random

def main():
    capitals = {'Alabama':'Montgomery', 'Alaska':'Juneau', 'Arizona':'Phoenix', 'Arkansas':'
        Little Rock' ... }

    correct = 0; wrong = 0; next_question = True; index = 0
    user_solution = ''

    while next_question:

        state_iterator = iter(capitals)

        index = (random.randint(1,50) - 1)
        for i in range (index-1):
            temp = state_iterator.__next__()
        current_state = str(state_iterator.__next__())
```

Ushtrimi #2 - Zgjidhja II

```
        user_solution = input(f'What is the capital of {current_state}? (or enter 0 to quit): ')
        if user_solution == '0':
            next_question = False
            print(f'You had {correct} correct responses and '
                  f'{wrong} incorrect responses.')
        elif user_solution == capitals[current_state]:
            correct = correct + 1
            print('That is correct.')
        else:
            wrong = wrong + 1
            print('That is incorrect.')

if __name__ == '__main__':
    main()
```

Ushtrimi #3 Enkriptim/Dekriptim skedari

Shkruani një program që:

- përdor një **fjalor** codes për ti caktuar çdo shkronje një kod (simbol tjetër), p.sh.:

```
codes = {'A': '%', 'a': '9', 'B': '@', 'b': '#', ...}|
```

- hap një file tekst (origjinali) dhe lexon përmbajtjen;
- për çdo karakter në file kërkon kodin përkatës në codes dhe e shkruan në një **file të dytë** (skedari i enkriptuar);
- pra çdo karakter në file-in e dytë është kodi i karakterit përkatës në file-in e parë.

Ushtrimi #3 - Zgjidhja (a)

```
CODE = {'A':')','a':'O','B':'(', ...}
def main():
    result = convert()
    output_name = input('Enter the name of the output file: ')
    output_file = open(output_name, 'w')
    output_file.write(result)
    output_file.close()

def convert():
    input_name = input('Enter the name of the input file: ')
    input_file = open(input_name, 'r')
    result = ''
    text = input_file.read()

    for ch in text:
        if ch.isspace(): result = result + ch
        else: result = result + CODE[ch]
    return result

if __name__ == '__main__':
    main()
```

Ushtrimi #3 Enkriptim/Dekriptim skedari

Shkruani edhe një program të dytë që:

- hap file-in e enkriptuar;
- përdor fjalorin e kundërt (ose një mënyrë tjetër) për të kthyer kodet në shkronjat origjinale;
- shfaq në ekran përmbajtjen e **dekriptuar**.

Ushtrimi #3 - Zgjidhja (b)

```
CODE = {'A':')','a':'0','B':'(','b':'9', ... }

def main():
    result = convert()
    print(result)

def convert():
    input_name = input('Enter the name of the input file: ')
    input_file = open(input_name, 'r')
    result = ''
    text = input_file.read()

    for ch in text:
        if ch.isspace(): result = result + ch
        else: result = result + CODE[ch]

    return result

if __name__ == '__main__':
    main()
```

Ushtrimi #4

Fjalë unike. Shkruaj një program që lexon një skedar dhe shfaq listën e të gjitha fjalëve unike të gjetur në atë skedar.

Ushtrimi #4 - Zgjidhja

```
def main():
    input_name = input('Enter the name of the input file: ')
    input_file = open(input_name, 'r')
    text = input_file.read()
    words = text.split()

    unique_words = set(words)
    print('These are the unique words in the text:')
    for word in unique_words:
        print(word)

    input_file.close()

if __name__ == '__main__':
    main()
```

Ushtrimi #5 Frekuenca e fjalëve

Shkruani një program që:

- lexon përmbajtjen e një **file-i tekst**;
- krijon një **fjalor** ku:
  - çelësat janë fjalët individuale të gjetura në file;
  - vlerat janë numri i herëve që shfaqet secila fjalë.
- p.sh., nëse fjala `"the"` shfaqet 128 herë, fjalori duhet të ketë elementin `{'the': 128}`.
- në fund:
  - ose shfaq në ekran frekuencën e çdo fjale,
  - ose krijon një **file të dytë** me listën e çdo fjale dhe frekuencën përkatëse (p.sh. `freq.txt`).

Ushtrimi #5 - Zgjidhja

```
def main():
    counter = {}
    input_name = input('Enter the name of the input file: ')
    input_file = open(input_name, 'r')
    text = input_file.read()
    words = text.split()

    unique_words = set(words)
    for word in unique_words: counter[word] = 0

    for item in words: counter[item] += 1

    print(f'{"Word":15}\t{"Occurrences":15}')
    print('-----------------------------------------------')
    while len(counter)>0:
        pair = counter.popitem()
        print(f'{pair[0]:15}{pair[1]:<15}')

if __name__ == '__main__':
    main()
```

Ushtrimi #6 Analiza e dy skedarëve

Shkruani një program që krahason dy **dy file tekst** duke përdorur *set*:

- krijo dy **bashkësi fjalësh**:

$$A = \{\text{fjalët në file-in 1}\}, \quad B = \{\text{fjalët në file-in 2}\};$$

- shfaq listën e **të gjitha fjalëve të ndryshme dhe të përbashkëta** që gjenden në të dy file-t:

$$A \cup B, \quad A \cap B$$

- shfaq listën e **fjalëve që janë vetëm në file-in e parë**:

$$A \setminus B$$

- shfaq listën e **fjalëve që janë vetëm në file-in e dytë**:

$$B \setminus A$$

- shfaq listën e **fjalëve që janë në njërin prej file-ve, por jo në të dy** (diferenca simetrike):

$$A \triangle B$$

Ushtrimi #6 - Zgjidhja I

```
def main():
    input_name = input('Enter the name of the first input file: ')
    file1 = open(input_name, 'r')
    text1 = file1.read()
    file1.close()
    words1 = text1.split()
    set1 = set(words1)

    input_name = input('Enter the name of the second input file: ')
    file2 = open(input_name, 'r')
    text2 = file2.read()
    file2.close()
    words2 = text2.split()
    set2 = set(words2)

    union = set1.union(set2)
    print('These are the unique words that are contained in both files:')
    for item in union: print(item)
    print()
```

Ushtrimi #6 - Zgjidhja II

```
intersection = set1.intersection(set2)
print('These are the words that appear both files:')
for item in intersection:
    print(item)
print()

difference1 = set1.difference(set2)
print('These are the words that appear in the first file but do not appear in the second
    file:')
for item in difference1:
    print(item)
print()
```

Ushtrimi #6 - Zgjidhja III

```
    difference2 = set2.difference(set1)
    print('These are the words that appear in the second file but do not appear in the first file:')
    for item in difference2: print(item)
    print()

    sym_diff = set1.symmetric_difference(set2)
    print('These are the words that appear in the first file or the second file but do not appear in the both files:')

    for item in sym_diff: print(item)
    print()

if __name__ == '__main__':
    main()
```

Ushtrimi #7 Fituesit e World Series

Në skedarin `WorldSeriesWinners.txt` gjendet një listë kronologjike e ekipeve fituese të World Series nga viti 1903 deri më 2009. Çdo rresht është emri i ekipit fitues për një vit të caktuar (ka rreshta të veçantë që tregojnë se në 1904 dhe 1994 nuk është luajtur).

Shkruani një program që:

- lexon file-in dhe ndërton:
  - një fjalor `team_wins`: {ekipi → numri i titujve};
  - një fjalor `year_champ`: {viti → ekipi fitues}.
- i kërkon përdoruesit një vit në intervalin **19032009**;
- nëse për atë vit nuk është luajtur World Series, shfaq një mesazh përkatës;
- përndryshe shfaq:
  - emrin e ekipit që ka fituar atë vit;
  - numrin total të titujve që ky ekip ka fituar (duke përdorur `team_wins`).

Përmbajtja e skedarit WorldSeriesWinners.txt

```
Boston Americans
New York Giants
Chicago White Sox
Chicago Cubs
Chicago Cubs
Pittsburgh Pirates
Philadelphia Athletics
...
```

Ushtrimi #7 - Zgjidhja I

```
BASE_YEAR = 1903

def main():
    year_dict = {}
    count_dict = {}

    input_file = open('WorldSeriesWinners.txt', 'r')
    winners = input_file.readlines()

    for i in range(len(winners)):
        team = winners[i].rstrip('\n')

        year = BASE_YEAR + i
        if year >= 1904: year += 1
        if year >= 1994: year += 1
        year_dict[str(year)] = team

        if team in count_dict: count_dict[team] += 1
        else: count_dict[team] = 1
```

Ushtrimi #7 - Zgjidhja II

```
    year = input('Enter a year in the range 1903-2009: ')

    if year == '1904' or year == '1994':
        print(f"The world series wasn't played in the year {year}.")
    elif year < '1903' or year > '2009':
        print(f'The data for the year {year} is not included in our database.')
    else:
        winner = year_dict[year]
        wins = count_dict[winner]
        print(f'The team that won the world series in {year} is the {winner}.')
        print(f'They won the world series {wins} times.')

if __name__ == '__main__':
    main()
```

Ushtrimi #8 Emra dhe adresa email

Shkruani një program që:

- mban një **fjalor** ku çelësat janë **emrat** dhe vlerat janë **adresat e email-it**:

```
{ "Ana":  "ana@example.com", ...}
```

- në çdo nisje të programit:
  - lexon nga një file binar (p.sh. `emails.dat`) dhe bën *unpickle* të fjalorit, nëse file-i ekziston.
- shfaq një **menu** që lejon përdoruesin të:
  - kërkojë adresën e email-it për një emër;
  - shtojë një emër dhe një adresë të re email-i;
  - ndryshojë adresën e email-it për një emër ekzistues;
  - fshijë një emër (dhe email-in përkatës) nga fjalori.
- kur përdoruesi zgjedh **dalje** nga programi:
  - bën *pickle* të fjalorit dhe e ruan në file.

Ushtrimi #8 - Zgjidhja I

```
import pickle

LOOK_UP = 1; ADD = 2; CHANGE = 3; DELETE = 4; QUIT = 5
FILENAME = 'emails.dat'

def main():
    emails = load_emails()
    choice = 0

    while choice != QUIT:

        choice = get_user_choice()
        if choice== LOOK_UP: look_up(emails)
        elif choice == ADD: add(emails)
        elif choice == CHANGE: change(emails)
        elif choice == DELETE: delete(emails)

    save_emails(emails)
    print('Information saved.')
```

Ushtrimi #8 - Zgjidhja II

```
def load_emails():
    try:
        input_file = open(FILENAME, 'rb')
        email_dict = pickle.load(input_file)
        input_file.close()

    except IOError:
        email_dict = {}

    return email_dict
```

Ushtrimi #8 - Zgjidhja III

```
def get_user_choice():
    print()
    print('Menu')
    print('----------------------------------------')
    print('1. Look up an email address')
    print('2. Add a new name and email address')
    print('3. Change an existing email address')
    print('4. Delete a name and email address')
    print('5. Quit the program')
    print()

    choice = int(input('Enter your choice: '))

    while choice < LOOK_UP or choice>QUIT:
        choice = int(input('The choice you entered is invalid. Please enter a valid choice: ')
            )

    return choice
```

Ushtrimi #8 - Zgjidhja IV

```
def look_up(emails):
    name = input('Enter a name: ')

    if name in emails:
        print(f' Name: {name}')
        print(f'Email: {emails[name]}')
    else:
        print('The specified name was not found.')
```

Ushtrimi #8 - Zgjidhja V

```
def add(emails):
    name = input('Enter name: ')
    address = input('Enter email address: ')

    if name not in emails:
        emails[name] = address
        print('Name and address have been added.')
    else:
        print('That name already exists.')
```

Ushtrimi #8 - Zgjidhja VI

```
def change(emails):
    name = input('Enter name: ')

    if name in emails:
        address = input('Enter the new address: ')
        emails[name] = address
        print('Information updated.')
    else:
        print('The specified name was not found.')
```

Ushtrimi #8 - Zgjidhja VII

```
def delete(emails):
    name = input('Enter name: ')

    if name in emails:
        del emails[name]
        print('Information deleted.')

    else:
        print('The specified name was not found.')
```

Ushtrimi #8 - Zgjidhja VIII

```
def save_emails(emails):
    output_file = open(FILENAME, 'wb')
    pickle.dump(emails, output_file)

    output_file.close()

if __name__ == '__main__':
    main()
```

Ushtrimi #9 Simulimi i Blackjack I

Shkruani një program që zgjeron programin `card_dealer.py` për të simuluar një version të thjeshtuar të lojës **Blackjack** mes dy lojtarëve virtualë.

Rregullat e vlerës së letrave:

- kartat numerike kanë vlerën e numrit të tyre (p.sh. 2 = 2 pikë, 5 = 5 pikë);
- *jack*, *queen*, *king* kanë vlerë **10** pikë;
- *ace* ka vlerë **1** ose **11** pikë.

Ushtrimi #9 Simulimi i Blackjack II

Rregullat e lojës:

- programi shpërndan letra për secilin lojtar, një nga një, nga kuleta;
- loja vazhdon derisa **dora e një lojtari kalon 21** pikë; në atë moment, lojtari tjetër është fituesi;
- mund të ndodhë që të dy lojtarët të kalojnë 21 njëkohësisht: në këtë rast, **nuk ka fitues**;
- procesi përsëritet derisa të jenë mbaruar të gjitha letrat në kuletë.

Vlera e asit:

- nëse një lojtar merr një *ace*, programi vendos:
  - nëse duke e numëruar si 11 pikë dora *nuk* kalon 21, atëherë as-i vlen **11**;
  - përndryshe, as-i vlen **1** pikë.

Ushtrimi #9 - Zgjidhja I

```
MAX = 21
def main():
    hand1 = 0
    hand2 = 0
    deck = create_deck()

    while hand1 <= MAX and hand2 <= MAX:
        card1, value1 = deck.popitem()
        hand1 = update_hand_value(hand1, value1, card1)

        card2, value2 = deck.popitem()
        hand2 = update_hand_value(hand2, value2, card2)

        print(f'Player 1 was dealt {card1}')
        print(f'Player 2 was dealt {card2}')
        print()
```

Ushtrimi #9 - Zgjidhja II

```
    if hand1 > MAX and hand2 > MAX:
        print("There is no winner.")
    elif hand1 > 21:
        print("Player 2 wins.")
    else:
        print("Player 1 wins.")
```

Ushtrimi #9 - Zgjidhja III

```
def create_deck():
    suits = ['Spades','Hearts','Clubs','Diamonds']
    special_values = {'Ace':1, 'King':10, 'Queen':10, 'Jack':10}

    numbers = ['Ace', 'King', 'Queen', 'Jack']
    for i in range(2,11):
        numbers.append(str(i))

    deck = {}
    for suit in suits:
        for num in numbers:
            # Values 2-10.
            if num.isnumeric():
                deck[num + ' of ' + suit] = int(num)
            # Ace, King, Queen, or Jack.
            else:
                deck[num + ' of ' + suit] = special_values[num]
    return deck
```

Ushtrimi #9 - Zgjidhja IV

```
def update_hand_value(hand, value, card):
    if not card.startswith('Ace'):
        return hand+value
    elif hand > 10:
        return hand + value
    else:
        return hand + 11

if __name__ == '__main__':
    main()
```

Ushtrimi #10 Indeksi i fjalëve

Shkruani një program që lexon përmbajtjen e një **file-i tekst** (p.sh. `Kennedy.txt`) dhe ndërton një **fjalor** ku:

- **Çelësat** janë fjalët individuale të gjetura në file;
- **Vlerat** janë lista e **numrave të rreshtave** ku shfaqet secila fjalë.

Shembull: nëse fjala `"robot"` gjendet në rreshtat 7, 18, 94 dhe 138, fjalori duhet të ketë elementin:

```
{"robot":  [7, 18, 94, 138]}
```

Pasi fjalori është ndërtuar, programi duhet të:

- krijojë një file të ri **indeksi** (p.sh. `index.txt`);
- shkruajë në të listën alfabetike të të gjitha fjalëve (çelësat e fjalorit);
- për secilën fjalë të shënojë numrat e rreshtave ku ajo shfaqet.

Shembull skedari: Kennedy.txt

```
We observe today not a victory
of party but a celebration
of freedom symbolizing an end
as well as a beginning
signifying renewal as well
as change
```

Shembull skedari: index.txt

```
end: 3
a: 1 2 4
party: 2
victory: 1
observe: 1
symbolizing: 3
but: 2
change: 6
as: 4 5 6
renewal: 5
signifying: 5
We: 1
freedom: 3
well: 4 5
an: 3
beginning: 4
celebration: 2
of: 2 3
today: 1
not: 1
```

Ushtrimi #10 - Zgjidhja I

```
def get_word_dict(line_list):
    count = 0
    word_dict = {}

    for e in line_list:
        words = e.split(' ')

        for w in words:
            if w in word_dict:
                word_dict[w].add(count + 1)
            else:
                word_dict[w] = set([count + 1])

        count += 1

    return word_dict
```

Ushtrimi #10 - Zgjidhja II

```
1  def write_index_file(word_dict):
2      outputfile = open('index.txt', 'w')
3
4      for key in word_dict:
5          outputfile.write(key + ': ')
6          for val in word_dict[key]:
7              outputfile.write(str(val) + ' ')
8          outputfile.write('\n')
9
10     outputfile.close()
```

Ushtrimi #10 - Zgjidhja III

```
def main():
    inputfile = open('Kennedy.txt', 'r')
    line_list = inputfile.readlines()
    inputfile.close()

    for i in range(len(line_list)):
        line_list[i] = line_list[i].rstrip('\n')

    word_dict = get_word_dict(line_list)

    write_index_file(word_dict)

if __name__ == '__main__':
    main()
```

Suksese!